BE-JV-005

Técnicas de Programação I

Professores

Vinicius Martins Cornieri
Arquiteto de Software

/vinicius-cornieri

Maringá - PR

/ViniciusCornieri

Nerd

ada

Apresentações

Técnicas de Programação I

→ Conteúdo Abordado | API de Datas, Streams, Programação Funcional e Concorrência.

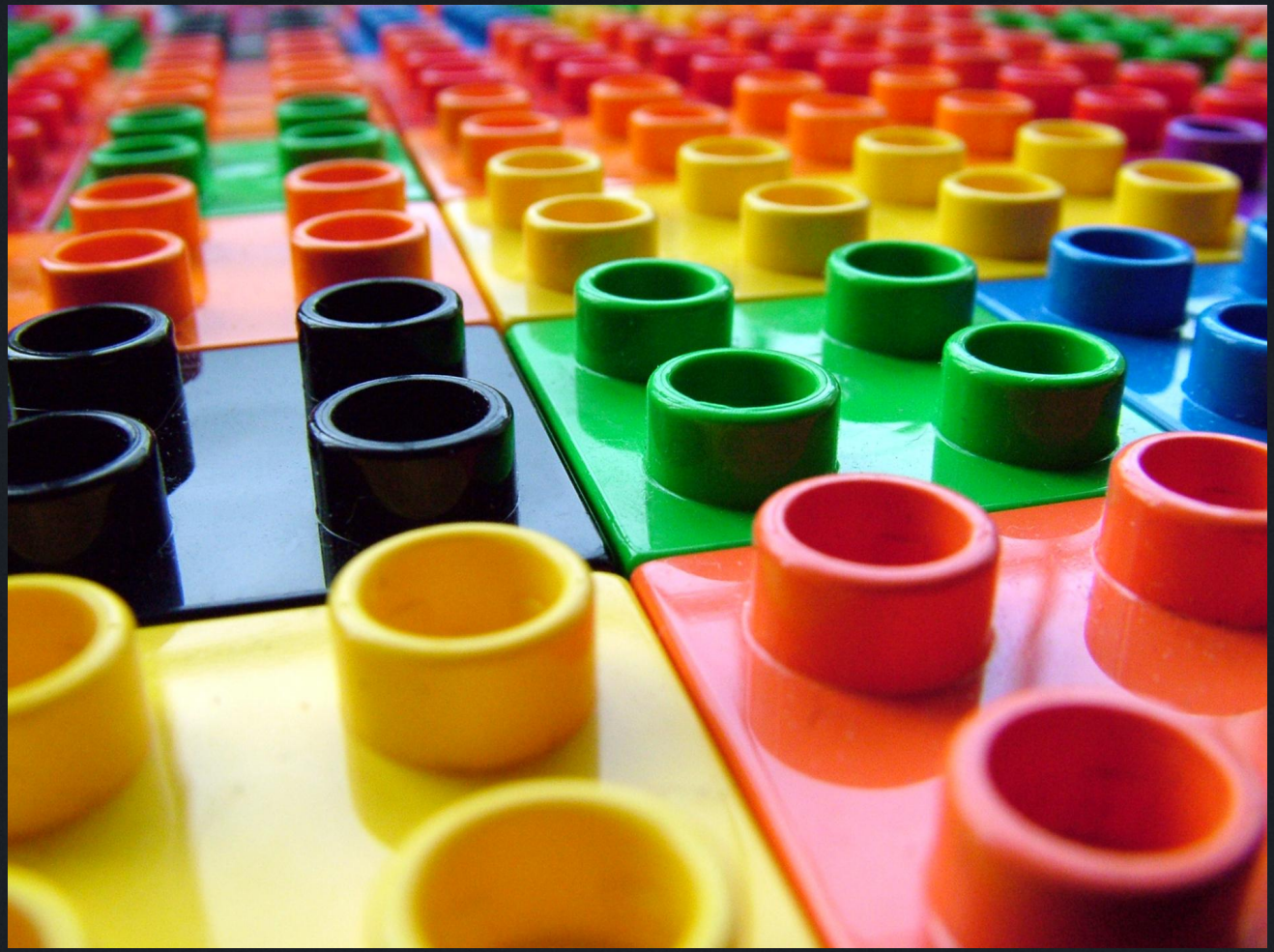
→ Dias e Horário | Terça e quinta das 19h00 às 22h00

→ Processo Avaliativo | Rubrica - Engajamento nas aulas, listas e Projeto

→ Acordos | Câmera ligada, participação ativa

ada

Building Blocks



Programação

ada

Ditado Familiar

> *“Tudo o que é DEMAIS, é BASTANTE!”*
>
> Moacir Martins

Datas -> java.util VS java.time

O Caos no fechamento dos CDBs

→ Temos um Job legado que roda diariamente que recentemente foi refatorado. Agora com a implantação o time de sustentação foi acionado indicando que os CDBs não estão sendo encerrados corretamente e continuam remunerando os clientes mesmo depois do esperado.

ada

java.time

Classe	O que representa?	Quando usar?
LocalDate	Apenas Data (Ano, Mês, Dia)	Data de nascimento, vencimento de boleto.
LocalTime	Apenas Hora (HH:mm:ss)	Horário de fechamento do pregão.
LocalDateTime	Data + Hora (Sem Fuso)	Agendamentos locais, sem dependência geográfica.
Instant	Momento específico representando quanto tempo passou desde 1970 UTC.	Auditoria e persistência em Banco de Dados.

java.time

Classe	O que representa?	Quando usar?
Period	Quantidade de tempo (Anos, Meses, Dias)	Calcular a idade de um utilizador ou tempo até uma data.
Duration	Quantidade de tempo (Horas, Minutos, Segundos)	Medir tempo de execução de uma task ou definir timeouts.

Fixação

Exercícios

TechKidsOpeningValidator.java

→ Vamos criar um novo produto a conta "Tech Kids", voltada para educação financeira infantil. A regra de negócio principal estipula que, para abrir esta conta, o titular não pode ter 18 anos completos ou mais na data do pedido. Implemente o validador de elegibilidade.

Exemplo:

```java
public class TechKidsOpeningValidator {

    public boolean isEligibleForTechKids(LocalDate birthDate, LocalDate requestDate){
        // TODO
        return false;
    }
}
```

ada

Exercícios

BoletoUpdater.java

→ Precisamos de um atualizador de boletos vencidos. Teremos um método que irá receber um boleto que possui, a data de vencimento prevista (*LocalDate*), a data de referência do pagamento atual, o valor original do boleto e aplique juros fixos e multa diária. Retorne o valor atualizado do boleto.

Exemplo:

```
Boleto {
    Vencimento: 2025-12-01,
    Pagamento: 2025-12-25,
    Valor original: R$ 1.000,00,
    Juros fixos: R$ 50,00,
    Multa diária: R$ 10,00
}
Valor atualizado: R$ 1.290,00
```

Exercícios

CalculadorDeParcelas.java

→ Agora para o nosso módulo de empréstimos precisamos gerar a data de cobrança das próximas parcela. Nossa programa irá receber a data do empréstimo, a quantidade de parcelas e o dia de pagamento selecionado entre os possíveis (5, 10, 15 ou 20) e irá retornar os dias de vencimento das parcelas iniciando no mês seguinte a tomada do empréstimo. Exemplo:

```
Empréstimo {
    criacao: 2026-02-02,
    parcelas: 2,
    diaDoVencimento: 15
}
Parcela 1: 2026-03-15
Parcela 2: 2026-04-15
```

ada

Exercícios

CalculadorDeParcelas.java

→ Foi solicitado uma melhoria no nosso sistema anterior de empréstimos para que ele ajuste a data da parcela para o próximo dia útil caso o vencimento caia em um final de semana.

Exemplo:

```
Empréstimo {
    criacao: 2026-02-02,
    parcelas: 2,
    diaDoVencimento: 15
}
Parcela 1: 2026-03-16
Parcela 2: 2026-04-15
```

ada

Exercícios

TokenValidator.java

→ Para esta atividade nos foi solicitado um validador de tokens JWT para as seguranças das nossas APIs. Nesse momento iremos focar em validar se o token está vencido.

Exemplo:

```
Token {
    iat: 1300819380,
    expireInMilis: 30000,
    issuer: “Vini”
}
```

Token válido/Token vencido

ada

Extras

Exercícios

Regulation1234Properties.java

→ Teremos uma nova feature no sistema porém pela legislação 1234 ela só será válida em produção em no dia 30/03/2026 às 00h e em homologação dia 15/02/2026 às 09h. Na nossa classe de configuração implemente o método *is1234Active(LocalDateTime currentDate)*

Exemplo:

```
Regulation1234Properties {
    productionEnvironment: true,
    productionChangeDateTime: LocalDateTime.of( ... ),
    homologationChangeDateTime: LocalDateTime.of( ... )

    public boolean is1234Active(LocalDateTime currentDate) {
        // TODO
    }

}
```

ada

Exercícios

ExtratoOrdenado.java

→ O sistema legado de banco de dados retorna os lançamentos misturados, fora de ordem. Precisamos garantir que a lista de lançamentos enviada para o aplicativo esteja ordenada cronologicamente pela data e hora da transação (do mais antigo para o mais recente). Na nossa classe de serviço, implemente o método sortTransactionsByDate (List<Transaction> transactions).

Exemplo:

```
public record Transaction(String description, LocalDateTime transactionDate, double
amount) {}

public class StatementService {

    public List<Transaction> sortTransactionsByDate(List<Transaction> transactions)
{
        // TODO: Ordenar a lista usando a data da transação (transactionDate)
        return transactions;
    }
```

Exercícios

ProcessadorDeTED.java

→ Ao contrário do PIX, as transferências via TED só podem ser processadas e liquidadas em dias úteis (Segunda a Sexta-feira) e dentro do horário comercial bancário (entre as 08:00 e às 16:59). Se um cliente tentar fazer um TED no fim de semana ou à noite, a operação deve ser agendada para a próxima data elegível. Cria o método que valida se o sistema pode processar o TED imediatamente caso não possa calcule a próxima data elegível.

ada

Obrig.ada